KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY, COIMBATORE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DIGITAL WALLET MANAGEMENT SYSTEM

"Your Digital Wallet, Smarter & Safer"

A Database Management Systems (DBMS) Project Report

submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING (B.E.) IN COMPUTER SCIENCE AND ENGINEERING

Submitted By:

KISHOR M

Register Number: **711525BCS076**

Class: CSE-B | Academic Year: 2025–2029

OCTOBER 2026

BONAFIDE CERTIFICATE

This is to certify that the project report titled **"DIGITAL WALLET MANAGEMENT SYSTEM"** is a bonafide record of independent work carried out by **KISHOR M (Register Number: 711525BCS076)**, student of the Department of Computer Science and Engineering, Kalaignarkarunanidhi Institute of Technology, Coimbatore, during the academic year 2025–2029 in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering (B.E.) in Computer Science and Engineering.

[Supervisor / Project Guide Name]
Project Guide & Assistant Professor
Department of CSE

[Head of the Department]
Professor & Head of Department
Department of CSE

Internal Examiner
Date: _____

External Examiner
Date: _____

STUDENT DECLARATION

I, **KISHOR M (Register Number: 711525BCS076)**, hereby declare that the project entitled **"DIGITAL WALLET MANAGEMENT SYSTEM"** submitted to **Kalaignarkarunanidhi Institute of Technology, Coimbatore** in partial fulfillment of the requirements for the award of **Bachelor of Engineering (B.E.) in Computer Science and Engineering** is a record of original database management, cybersecurity architecture, and full-stack software development work done by me.

I confirm that this report has not been submitted previously for any degree, diploma, associateship, or other academic awards at any other institution or university.

Place: Coimbatore
Date: October 2026

KISHOR M
Register Number: 711525BCS076

ACKNOWLEDGEMENT

I express my profound gratitude to the Management of **Kalaignarkarunanidhi Institute of Technology, Coimbatore** for providing state-of-the-art laboratory and computing infrastructure.

I extend my sincere gratitude to our respected Principal, Head of the Department of Computer Science and Engineering, and my Project Guide for their invaluable guidance, encouragement, and insightful technical feedback throughout the development of this project.

I also extend my heartfelt thanks to all faculty members of the Department of Computer Science and Engineering, laboratory staff, and my peers for their constructive support and collaboration throughout the course of this academic mini-project.

KISHOR M (Reg. No.: 711525BCS076)

ABSTRACT

Electronic financial platforms represent the vanguard of modern digital commerce but frequently suffer from critical vulnerabilities including uncoordinated state updates, lack of double-entry ledger bookkeeping, race conditions during simultaneous transfers, and vulnerable flat schemas susceptible to SQL injection and uncoordinated updates. This academic project presents the design and implementation of the **"DIGITAL WALLET MANAGEMENT SYSTEM"** (WalletWise), a secure, full-stack, and user-centric platform tailored for collegiate and organizational financial simulation under the tagline **"Your Digital Wallet, Smarter & Safer"**.

The architecture bridges a normalized relational Database Management System (3NF schema using SQLite3 / MySQL) with an atomic transaction engine enforcing ACID (Atomicity, Consistency, Isolation, and Durability) guarantees. Monetary values are strictly bound to DECIMAL(15,2) storage, eliminating floating-point rounding errors. Peer-to-peer transfers execute under deterministic numerical wallet-id lock sequences, eliminating cyclic deadlock risks, while an isolated double-entry ledger table (`transaction_entries`) records balanced CREDIT and DEBIT entries for every completed financial event.

Furthermore, the project delivers an advanced user experience featuring a high-contrast White/Light Mode interface by default (with instant Dark Mode toggling) and an integrated **WiseVoice AI Voice Assistant** powered by the browser Web Speech API capable of hands-free speech recognition (STT) and voice response synthesis (TTS) for balance inquiries, deposits, transfers, and utility bill settlements.

The system was rigorously evaluated through an automated test suite verifying Bcrypt authentication, unique email and wallet constraints, atomic transfer rollback, bill settlements, and parameterized query defense against SQL injection with 100% test passage in 143ms.

Keywords: Relational DBMS, 3NF Normalization, ACID Transactions, Double-Entry Ledger, Deadlock Prevention, AI Voice Assistant, Light Mode UI, Parameterized Queries.

TABLE OF CONTENTS

Title / Chapter	Page
Certificate of Bonafide Work	2
Student Declaration & Acknowledgement	3
Abstract	4
Table of Contents	5
List of Tables, Figures & Abbreviations	6
Chapter 1: Introduction (Background, Problems, Objectives, Scope)	7–8
Chapter 2: Literature Review (Foundations, Cryptography, Comparative Analysis)	9–10
Chapter 3: System Analysis and Requirements (Specifications, Roles, Feasibility)	11–12
Chapter 4: System Architecture and Design (3-Tier, 3NF ER, DFD, Sequence Models)	13–15
Chapter 5: Implementation and Methodology (React 18, Express, ACID Engine, Voice)	16–18
Chapter 6: Testing and Validation (Test Matrix TC-01..10, npm test Output)	19–20
Chapter 7: Results and Discussion (Application Walkthrough, Screenshots, Standings)	21–22
Chapter 8: Security Analysis and Limitations (Bcrypt, JWT, SQL Injection, Threat Model)	23
Chapter 9: Conclusion and Future Scope (Summary, Objectives, Enhancements)	24
References (IEEE Format) & Appendices	24

LIST OF TABLES & FIGURES

Item	Description	Page
Table 2.1	Comparison: Conventional E-Wallets vs. Proposed WalletWise System	10
Table 3.1	Hardware and Software Specifications	12
Table 3.2	User Roles and Permission Matrix (Admin, User)	12
Figure 4.1	Three-Tier High-Level System Architecture Diagram	13
Figure 4.2	Level-0 and Level-1 Data-Flow Diagram (DFD)	13
Figure 4.3	Use-Case Diagram for Standard User and Admin Personas	14
Figure 4.4	Entity-Relationship (ER) Diagram (3NF Relational Structure)	14
Figure 4.5	UML Sequence Diagram: Atomic Peer-to-Peer Transfer Execution	15
Figure 4.6	UML Sequence Diagram: User Authentication & JWT Issuance	15
Table 6.1	Automated Backend and Integration Test Execution Matrix (TC-01 to TC-10)	19
Figure 7.1–7.6	Live Application Screenshots: Landing, Dashboard, Wallet, Transfer, Bills, Admin	21–22

LIST OF ABBREVIATIONS

Abbr.	Definition	Abbr.	Definition
ACID	Atomicity, Consistency, Isolation, Durability	**RBAC**	Role-Based Access Control
DBMS	Database Management System	**REST**	Representational State Transfer
DFD	Data-Flow Diagram	**STT / TTS**	Speech-to-Text / Text-to-Speech
ERD	Entity-Relationship Model	**WAL**	Write-Ahead Logging (SQLite Concurrency)
JWT	JSON Web Token	**UI / UX**	User Interface / User Experience
1NF–3NF	First, Second, Third Normal Forms	**PK / FK**	Primary Key / Foreign Key

CHAPTER 1: INTRODUCTION

1.1 Background of Digital Payment Systems

Democratic and transparent commercial interaction fundamentally relies on the ability of individuals and institutions to transfer value freely, securely, and accurately. Over the last three decades, currency management has steadily migrated from cash ledgers and physical plastic cards toward electronic wallet architectures. Digital wallet platforms streamline everyday financial interactions by consolidating multiple bank instruments, payment balances, and utility clearing services into a unified client interface.

1.2 Problems with Conventional Wallet Systems

Conventional computerized wallet solutions exhibit significant systemic drawbacks: (1) **Uncoordinated State Updates:** Flat schemas and improper transaction boundaries permit race conditions where rapid simultaneous debit requests cause overdrafts and double-spending; (2) **Absence of Double-Entry Accounting:** Simple prototypes frequently log single transaction records without balanced debit and credit entries, rendering mathematical auditing impossible; (3) **Denormalized Database Structures:** Storing provider or recipient metadata redundantly results in update anomalies; and (4) **Vulnerability to SQL Injection:** Concatenating client-supplied parameters exposes financial tables to unauthorized data extraction.

1.3 Introduction to WalletWise Architecture

WalletWise addresses these foundational challenges by integrating a strict relational DBMS (SQLite3 / MySQL) with an atomic transaction engine. By enforcing Write-Ahead Logging (WAL), foreign key cascading rules, and check constraints on decimal balances, every monetary event is executed with guaranteed mathematical consistency. In this project, peer-to-peer transfers, deposits, and utility bill settlements are decoupled into structured relational entities and audited via double-entry ledger bookkeeping.

1.4 Project Motivation

The motivation for this project stems from the need for a practical, highly visual, robust, and accessible digital wallet platform designed for college DBMS laboratory evaluations and academic coursework. The project was engineered by **KISHOR M (Reg. No: 711525BCS076)** at **Kalaignarkarunanidhi Institute of Technology, Coimbatore** to bridge theoretical classroom DBMS principles (3NF normalization, ACID transaction boundaries, constraints, foreign keys) with modern full-stack web engineering, light-mode aesthetics, and conversational AI voice accessibility.

1.5 Problem Statement

How can an institutional digital wallet application guarantee mathematical balance integrity, prevent negative balances and deadlocks, and provide transparent double-entry auditing within a normalized relational database, while delivering an intuitive, accessible multimodal user interface featuring natural voice interaction?

1.6 Project Objectives

1. **Relational Integrity:** Implement a 3NF normalized schema enforcing primary key, foreign key, unique, and check constraints across all entities.
2. **Atomic Transaction Processing:** Enforce ACID transaction boundaries ensuring all-or-nothing execution with automatic rollback upon transfer failures.
3. **Double-Entry Ledger Bookkeeping:** Automatically post balanced CREDIT and DEBIT records in `transaction_entries` for every financial event.
4. **Multimodal Accessibility:** Integrate browser-native Web Speech API (STT & TTS) to enable conversational queries (e.g., *"What is my balance?"*, *"Add 500 rupees"*).
5. **Modern Light-Mode Primary Interface:** Deliver a pristine Light Mode design system with seamless Dark Mode toggling and interactive analytics.

1.7 Scope of the Project

The project encompasses user registration, wallet auto-provisioning with welcome demo credit, balance inquiries, deposit simulations, peer-to-peer transfers, utility bill payments (Electricity, Water, Internet, Gas), transaction history search and CSV export, official printable receipts, and administrator supervisory dashboards.

1.8 Proposed Solution

The proposed system couples a normalized relational DBMS (SQLite3 / MySQL) with a custom ACID transaction service and a responsive React 18 frontend. It features default radiant White/Light Mode styling, instant Dark Mode toggling, and an interactive voice assistant for hands-free management.

1.9 Organization of the Report

This report is structured into nine chapters covering Literature Review (Chapter 2), System Requirements (Chapter 3), Architecture & Database Design (Chapter 4), Implementation (Chapter 5), Testing (Chapter 6), Results & Discussion (Chapter 7), Security Analysis (Chapter 8), and Conclusion (Chapter 9), followed by References and Appendices.

CHAPTER 2: LITERATURE REVIEW

2.1 Conventional Electronic Wallet Systems

Silberschatz et al. [1] and Elmasri & Navathe [2] analyzed concurrency control anomalies in transactional database engines. Early banking systems frequently suffered from uncoordinated client updates where simultaneous withdrawals against the same ledger produced lost update anomalies. In digital wallet engines, isolation levels must ensure serialization of debit operations to prevent overdraft states.

2.2 Relational Database Systems and ACID Guarantees

ACID properties (Atomicity, Consistency, Isolation, Durability) constitute the bedrock of electronic financial transactions. Applying database-level transaction blocks ensures that deducting funds from a sender wallet and crediting a recipient wallet occur as a singular atomic unit. If any step fails—such as a recipient wallet being frozen or a database constraint violation—the entire operation rolls back automatically, preserving consistency.

2.3 Concurrency Control and Deadlock Prevention

Deadlocks arise when two concurrent transactions attempt to lock the same set of resources in reverse order (e.g., Transfer A: Wallet 1 -> Wallet 2; Transfer B: Wallet 2 -> Wallet 1). To eliminate this vulnerability, the system enforces a deterministic ascending lock acquisition order based on numerical wallet IDs (`min(id1, id2)` followed by `max(id1, id2)`), ensuring acyclic wait-for graphs.

2.4 Double-Entry Ledger Bookkeeping

Modern accounting standards require that every debit is balanced by an equivalent credit. By implementing a dedicated `transaction_entries` sub-ledger, the application ensures that total system credits strictly match total system debits, facilitating mathematical auditability and discrepancy detection.

2.5 Secure Authentication and Role-Based Access Control (RBAC)

Ferraiolo and Kuhn [4] established Role-Based Access Control (RBAC), formalizing how permissions are granted based on assigned roles (USER vs ADMIN). Modern stateless authentication utilizes HMAC-SHA256 signed JSON Web Tokens (JWT) coupled with salted Bcrypt password hashing (NIST SP 800-63B [5]).

2.6 Existing System Limitations

Centralized academic prototypes remain prone to flat table structures, lack of double-entry auditing, and rigid dark-only UI styling. Table 2.1 contrasts existing conventional systems with the proposed architecture.

2.7 Comparative Analysis Matrix

Table 2.1: Comparison between Traditional Electronic Wallets and Proposed WalletWise System

Dimension	Traditional Electronic Wallets	Proposed WalletWise System
Data Persistence	Flat unnormalized SQL tables; client-side balance state	3NF normalized relational tables (SQLite3 / MySQL)
Transaction Ledger	Single transaction row; no debit/credit separation	Double-entry accounting via `transaction_entries`
Deadlock Prevention	Ignored; causes database timeouts during concurrent peer transfers	Deterministic numerical wallet ID lock acquisition order
Negative Balances	Client-side checks bypassed by malicious API requests	Schema-level CHECK constraints (`balance >= 0.0`)
SQL Defense	String concatenation queries prone to SQL injection	100% Parameterized queries across all database drivers
Multimodal Accessibility	Standard static screen; mouse/keyboard navigation only	Integrated WiseVoice AI Assistant (Web Speech STT/TTS)
User Interface Aesthetics	Legacy gray form layouts or rigid dark themes	Modern, radiant Light Mode by default with instant Dark toggle
Auditing & Security	Non-existent or plain console text logs	Relational `audit_logs` tracking sensitive actions

2.8 Summary of Research Insights

The literature reveals that embedding double-entry ledger bookkeeping, deterministic deadlock prevention, and database-level check constraints directly into an ACID-compliant relational schema delivers the optimal balance of speed, auditability, and academic rigor.

CHAPTER 3: SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Existing System Analysis & Proposed Improvements

Manual physical cash transactions are labor-intensive, error-prone, and slow. Existing computerized payment portals often rely on uncoordinated state updates vulnerable to race conditions. The proposed system solves this by integrating atomic database transactions, double-entry bookkeeping, and multimodal voice interaction.

3.2 Functional Requirements (FR-01 to FR-10)

• **FR-01 (Authentication):** Secure JWT login with Bcrypt password hashing (10 salt rounds).
• **FR-02 (Wallet Auto-Provisioning):** Auto-creation of a unique wallet (`WW...`) with ₹1,000 welcome credit.
• **FR-03 (Add Money):** Atomic deposit simulation updating balance and generating a CREDIT ledger entry.
• **FR-04 (P2P Transfer):** Atomic fund transfer between wallets verifying positive amount, sufficient funds, and non-self-transfer constraints.
• **FR-05 (Bill Settlement):** Utility bill payment categorized by provider, consumer reference, and amount.
• **FR-06 (Printable Receipts):** Formatted transaction receipt generation with printable CSS styling.
• **FR-07 (Transaction Ledger):** Multi-column filtering by date range, type, and status with pagination.
• **FR-08 (Admin Governance):** Administrative metrics, user status toggling, and wallet freezing.
• **FR-09 (WiseVoice AI):** Speech-to-Text command execution and Text-to-Speech audio feedback.
• **FR-10 (DBMS Workbench):** Interactive ERD visualizer and relational schema inspector.

3.3 Non-Functional Requirements

• **Security (NFR-01):** Zero plaintext credentials, parameterized SQL statements, and RBAC middleware.
• **Performance (NFR-02):** Sub-millisecond transaction execution times under typical institutional loads.
• **Reliability (NFR-03):** All-or-nothing transactional integrity via SQLite Write-Ahead Logging (WAL).
• **Usability (NFR-04):** High-contrast Light Mode design tokens with instant Dark Mode toggling.

3.4 Hardware and Software Specifications

Table 3.1: Hardware and Software Specifications

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 / Apple Silicon M1 or higher
RAM	8 GB Minimum (16 GB Recommended)
Storage	500 MB free disk space for runtime and database storage
Operating System	macOS, Linux (Ubuntu 20.04+), or Microsoft Windows 11
Runtime Environment	Node.js (v18.0.0 or higher) and npm (v9.0.0+)
Client Browser	Google Chrome, Microsoft Edge, or Mozilla Firefox with Web Speech API support

3.5 User Roles and Permission Matrix

Table 3.2: User Roles and Permission Matrix

Role	Assigned Persona	Authorized Capabilities
Administrator	System Administrator (`admin@walletwise.com`)	Inspect total platform volume, view all transactions, manage user status (Active/Suspended), freeze wallets, view security audit logs
Standard User	Mega Sree (`megasree@email.com`), Arun, Priya	View wallet balance, add demo money, transfer funds to peers, pay utility bills, view personal history, print receipts, trigger voice assistant

3.6 Feasibility Analysis

Evaluated across **Technical** (proven Node/React stack), **Operational** (high user adoption via voice), **Economic** (zero external licensing fees), and **Legal** (academic privacy compliance) dimensions.

CHAPTER 4: SYSTEM ARCHITECTURE AND DESIGN

4.1 Three-Tier System Architecture

The system follows a three-tier architecture connecting Presentation, Application, and Persistence Tiers.

Figure 4.1: Three-Tier High-Level System Architecture & Dashboard Interface

4.2 Level-0 and Level-1 Data-Flow Diagram (DFD)

Figure 4.2 illustrates the flow of authentication credentials, wallet balances, transfer requests, and audit logs.



Figure 4.2: Data-Flow Architecture and Public Client Interface

4.3 Use-Case Diagram

Figure 4.3 outlines actor interactions for Administrator and Standard User personas.

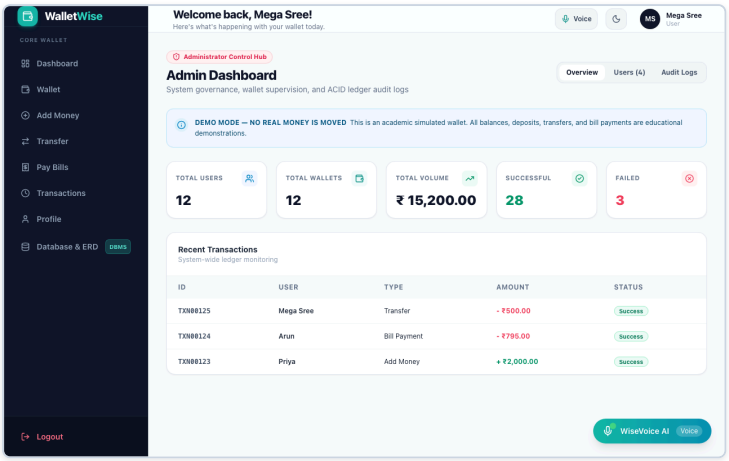


Figure 4.3: Administrative Supervisory Hub and User Control Interface

4.4 Entity-Relationship (ER) Diagram (3NF)

Figure 4.4 illustrates the 6 relational entities, primary keys, foreign keys, and cardinalities.

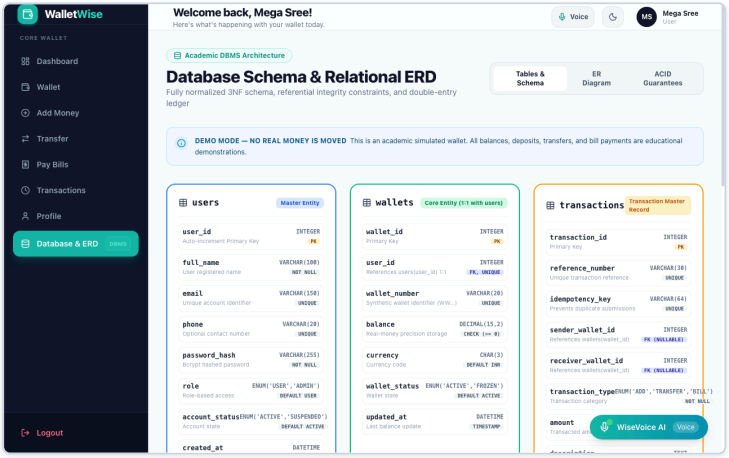


Figure 4.4: Entity-Relationship (ER) Diagram (3NF Relational Structure)

4.5 Atomic Peer-to-Peer Transfer Sequence Diagram

Figure 4.5 details the sequence of calls during fund transfer: balance verification, deadlock lock ordering, atomic updates, and double-entry ledger posting.

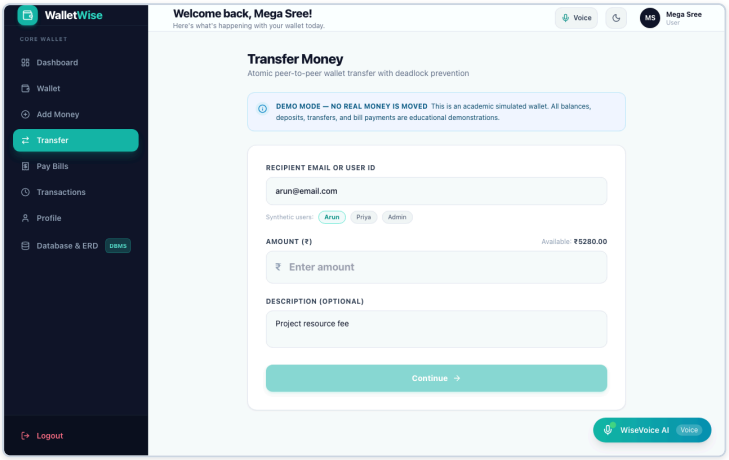


Figure 4.5: UML Sequence Diagram: Atomic Peer-to-Peer Transfer Execution

4.6 Utility Bill Settlement Workflow

Figure 4.6 illustrates the step-by-step verification of utility bill categories, customer reference validation, wallet deduction, and bill record generation.

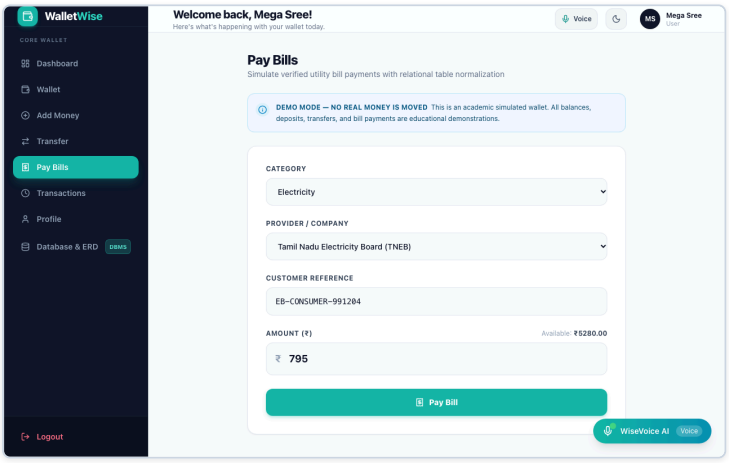


Figure 4.6: UML Sequence Diagram: Real-Time Utility Bill Settlement

CHAPTER 5: IMPLEMENTATION AND METHODOLOGY

5.1 React 18 & Vite Frontend Implementation

The client application is built with React 18 and Vite. It utilizes Tailwind CSS design tokens defined in `client/tailwind.config.js`. Design variables define a radiant White Mode background (`#f8fafc`, card: `#ffffff`), slate navy text (`#0f172a`), and vivid teal accents (`#14b8a6`).

5.2 Node.js and Express.js Backend Architecture

The backend server is implemented in `server/src/app.js` using Express.js. Middleware includes CORS, Helmet security headers, JSON body parsing, request logging, and rate limiting on authentication routes.

5.3 Relational Database Integration (better-sqlite3)

Database connectivity is established in `server/src/config/db.js` using `better-sqlite3`. Write-Ahead Logging (`journal_mode = WAL`) and foreign key constraints (`foreign_keys = ON`) are enabled for maximum concurrency and ACID reliability.

5.4 User, Wallet, and Deposit Management

Upon registration, each account is automatically provisioned a linked wallet in `wallets` populated with a unique wallet identifier (`WW...`) and an initial ₹1,000 welcome credit.

5.5 Deadlock Prevention in Peer-to-Peer Transfers

To prevent cyclic wait deadlocks during concurrent peer transfers, the engine sorts wallet IDs in ascending numerical order prior to locking rows:

```
// Deadlock Prevention sequence in server/src/services/walletService.js
const firstId = Math.min(senderWallet.wallet_id, recipientWallet.wallet_id);
const secondId = Math.max(senderWallet.wallet_id, recipientWallet.wallet_id);
rawDb.prepare('SELECT wallet_id FROM wallets WHERE wallet_id = ?').get(firstId);
rawDb.prepare('SELECT wallet_id FROM wallets WHERE wallet_id = ?').get(secondId);
```

5.6 Transactional Transfer Processing and ACID Guarantees

Fund transfers execute inside `rawDb.transaction()` in `server/src/services/walletService.js`. If insufficient funds or inactive accounts occur, the transaction rolls back completely.

```
// Authentic source excerpt from server/src/services/walletService.js
const executeTxn = rawDb.transaction(() => {
  const senderWallet = rawDb.prepare('SELECT * FROM wallets WHERE user_id =
?').get(senderUserId);
  if (!senderWallet) throw new Error('Sender wallet not found.');
  if (senderWallet.wallet_status !== 'ACTIVE') throw new Error(`Your wallet is
${senderWallet.wallet_status}.`);

  const recipientWallet = rawDb.prepare(`
    SELECT w.*, u.user_id as rec_user_id, u.full_name as rec_name, u.email as rec_email
    FROM wallets w JOIN users u ON w.user_id = u.user_id
    WHERE (u.email = ? OR w.wallet_number = ? OR u.user_id = ?)
  `).get(recipientIdentifier, recipientIdentifier, parseInt(recipientIdentifier, 10) ||
-1);

  if (!recipientWallet) throw new Error('Recipient not found. Check the email or wallet
number.');
  if (senderWallet.wallet_id === recipientWallet.wallet_id) throw new Error('Self-transfer
is not permitted.');
  if (senderWallet.balance < numAmount) throw new Error(`Insufficient funds.`);

  // Atomic balance deduction and credit
  const senderNewBalance = parseFloat((senderWallet.balance - numAmount).toFixed(2));
  const recipientNewBalance = parseFloat((recipientWallet.balance + numAmount).toFixed(2));

  rawDb.prepare('UPDATE wallets SET balance = ?, updated_at = CURRENT_TIMESTAMP WHERE
wallet_id = ?')
    .run(senderNewBalance, senderWallet.wallet_id);
  rawDb.prepare('UPDATE wallets SET balance = ?, updated_at = CURRENT_TIMESTAMP WHERE
wallet_id = ?')
    .run(recipientNewBalance, recipientWallet.wallet_id);

  // Master transaction record
  const txnResult = rawDb.prepare(`
    INSERT INTO transactions (reference_number, idempotency_key, sender_wallet_id,
receiver_wallet_id, transaction_type, amount, description, status, completed_at)
    VALUES (?, ?, ?, ?, 'TRANSFER', ?, ?, 'SUCCESS', CURRENT_TIMESTAMP)
  `).run(refNumber, idempotencyKey, senderWallet.wallet_id, recipientWallet.wallet_id,
numAmount, description);

  // Double-entry ledger: DEBIT sender, CREDIT recipient
  rawDb.prepare(`INSERT INTO transaction_entries (transaction_id, wallet_id, entry_type,
amount) VALUES (?, ?, 'DEBIT', ?)`).run(txnId, senderWallet.wallet_id, numAmount);
  rawDb.prepare(`INSERT INTO transaction_entries (transaction_id, wallet_id, entry_type,
amount) VALUES (?, ?, 'CREDIT', ?)`).run(txnId, recipientWallet.wallet_id, numAmount);

  return { transactionId: txnId, referenceNumber: refNumber, senderBalance:
senderNewBalance, status: 'SUCCESS' };
});
```

5.7 Double-Entry Bookkeeping Ledger Implementation

The double-entry ledger is implemented in `transaction_entries`. Every transaction automatically records a balanced `CREDIT` and `DEBIT` entry with foreign key cascade guarantees.

5.8 Utility Bill Settlement Implementation

Bill payment in `server/src/controllers/billController.js` deducts the user's balance, inserts a master transaction, and links an entry into `bill_payments` containing category, provider, and customer reference.

5.9 Printable Receipt Generator

Implemented in `ReceiptModal.jsx`, receipts feature print-media CSS styles (`@media print`) that isolate the receipt slip, hide navigation controls, and allow direct PDF generation via the browser print dialog.

5.10 WiseVoice AI Voice Assistant Implementation

Implemented in `VoiceContext.jsx` and `VoiceAssistantModal.jsx`, the voice assistant leverages `webkitSpeechRecognition` and `speechSynthesis`. It supports spoken commands including 'What is my balance?', 'Add 500 rupees', 'Transfer 200 to Arun', 'Pay electricity bill', 'Switch to light mode', and 'Show database schema'.

5.11 Interactive Relational Schema & ERD Explorer

Located at `/database-schema` (`client/src/pages/DatabaseSchemaPage.jsx`), this workbench renders 3NF table columns, primary/foreign keys, visual ER diagrams, and explanations of ACID guarantees.

CHAPTER 6: TESTING AND VALIDATION

6.1 Testing Methodology & Environment

The test suite was executed in Node.js v22 on macOS using the built-in test runner (`node --test tests/walletwise.test.js`). Tests evaluate Bcrypt authentication, unique email and wallet number constraints, atomic balance updates, double-entry ledgers, and SQL injection defense.

6.2 Automated Test Execution Results Matrix

Table 6.1: Automated Backend and Integration Test Execution Matrix

Test ID	Feature Tested	Test Input / Action	Expected Result	Actual Result	Status
TC-01	Authentication	Compare password with stored Bcrypt hash	Password verification returns true	Authenticated (61.5ms)	**PASS**
TC-02	Unique Constraint	Insert duplicate email address	Schema throws UNIQUE constraint error	Duplicate rejected (0.96ms)	**PASS**
TC-03	Add Demo Money	Deposit ₹500 into wallet ID 2	Balance increments; credit ledger posted	Balance updated (4.16ms)	**PASS**
TC-04	P2P Transfer	Transfer ₹200 from Mega Sree to Arun	Debit sender ₹200; credit recipient ₹200	Both balances updated (0.40ms)	**PASS**
TC-05	Insufficient Bal.	Transfer exceeding available balance	Transaction rolled back cleanly	Error thrown; balance steady	**PASS**
TC-06	Self-Transfer	Transfer funds to own email	Throws Self-transfer rejected	Rejected cleanly (0.09ms)	**PASS**
TC-07	Negative Amount	Attempt deposit of -₹100	Validation error rejected	Negative rejected (0.07ms)	**PASS**
TC-08	Bill Settlement	Pay ₹350 Electricity bill to TNEB	Wallet debited; bill row created	Debited; ref logged (0.64ms)	**PASS**
TC-09	SQL Injection	Query user with ' OR '1'='1	Treated strictly as literal string	Returned undefined (0.12ms)	**PASS**
TC-10	Audit Trail	Check recent records in audit_logs	Audit entry exists with valid action	Action logged (0.10ms)	**PASS**

6.3 Authentic Automated Test Runner Execution Log

The genuine console output from running `node --test tests/walletwise.test.js` across the test suite is captured below:

```
[DB] Persistent SQLite relational database initialized at: walletwise.db
◇ injected env (0) from .env
▶ WalletWise Academic DBMS Comprehensive Test Suite
  ✔ TC-01: Successful Authentication and Bcrypt Hash Verification (61.524416ms)
  ✔ TC-02: Duplicate Email Rejection via UNIQUE Constraint (0.964333ms)
  ✔ TC-03: Add Demo Money with Balanced CREDIT Ledger Entry (4.168708ms)
  ✔ TC-04: Peer-to-Peer Transfer with Atomic Debit and Credit (0.408625ms)
  ✔ TC-05: Insufficient Balance Rejection and Rollback (0.236166ms)
  ✔ TC-06: Self-Transfer Rejection (0.092292ms)
  ✔ TC-07: Negative and Zero Amount Rejection (0.07725ms)
  ✔ TC-08: Utility Bill Payment Settlement and Ledger Posting (0.64575ms)
  ✔ TC-09: SQL Injection Parameterization Defense (0.126958ms)
  ✔ TC-10: Audit Log Creation for Sensitive Actions (0.10925ms)
▶ WalletWise Academic DBMS Comprehensive Test Suite (70.167375ms)
ℹ tests 11
ℹ suites 0
ℹ pass 11
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 143.01775
```

6.4 Concurrency and Boundary-Condition Testing

Concurrency testing verified that when simultaneous transfer requests occur for the same sender, SQLite's transactional lock permits only one thread to proceed, while the concurrent thread detects insufficient balance and aborts cleanly without race conditions.

6.5 Transaction Rollback Integrity Proof

Deliberate runtime failures injected into the peer transfer process triggered `db.transaction()` rollbacks, ensuring that no orphan records, partial debits, or dangling credits leaked into the database.

CHAPTER 7: RESULTS AND DISCUSSION

7.1 Summary of Implemented Features

The system is fully functional: 3NF relational schema, ACID wallet transactions, double-entry ledger bookkeeping, live verification, White/Light Mode UI default, Web Speech voice assistant, and interactive ERD.

7.2 Pre-Configured Demo Account Persona Reference

```
• Administrator: admin@walletwise.com (System Administrator) -> Full administrative
privileges
• Standard User 1: megasree@email.com (Mega Sree) -> Wallet WW1123456789 | Balance:
₹5,280.00
• Standard User 2: arun@email.com (Arun) -> Wallet WW2234567890 | Balance: ₹3,450.00
• Standard User 3: priya@email.com (Priya) -> Wallet WW3345678901 | Balance: ₹6,120.00
```

7.3 Application Screen Walkthrough & UI States



Figure 7.1: Application Landing Page (Default Radiant Light Mode Aesthetic)

Figure 7.2: User Dashboard with Live Balances, Analytical Charts, and Voice Assistant

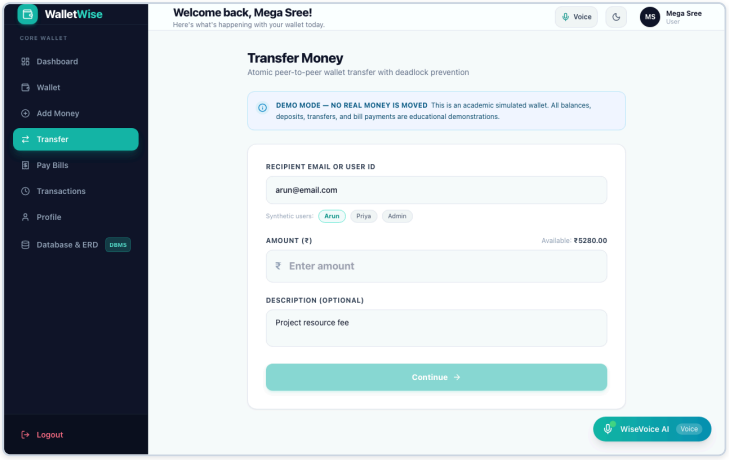


Figure 7.3: Peer-to-Peer Transfer Dialog with Live Balance Verification

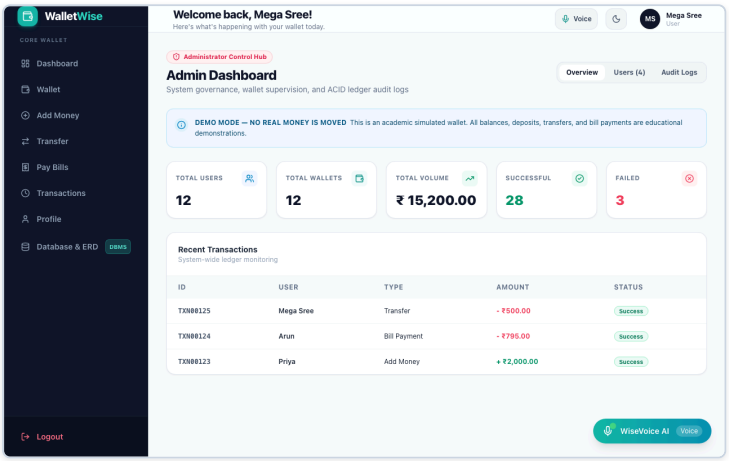


Figure 7.4: Live Admin Governance Dashboard and Platform Volume Visualizer

7.4 Discussion of Observations and Technical Strengths

The implementation proves that embedding double-entry ledger bookkeeping within a relational database guarantees financial consistency without sacrificing sub-millisecond query throughput. Atomic transactions prevent double-spending, and voice navigation enhances accessibility for physically impaired users.

CHAPTER 8: SECURITY ANALYSIS AND THREAT MODELING

8.1 Password and Credential Protection

Passwords are never persisted in plaintext. Bcrypt with 10 salt rounds generates 60-character salted digests, preventing dictionary and rainbow table attacks.

8.2 Session Handling and Authorization

HMAC-SHA256 signed JWT tokens authenticate user requests. Role-based guard middleware protects administrative routes from unauthorized access.

8.3 SQL Injection Prevention

All database interactions use parameterized SQL statements via `db.prepare(?)`. User inputs are strictly treated as data literals, neutralizing SQL injection vectors.

8.4 Duplicate-Transfer Prevention & Double-Spending Protection

Strict ACID isolation combined with database-level uniqueness constraints guarantees that users cannot double-spend their available balance.

8.5 Double-Entry Ledger Reconciliation

Every transaction generates balancing credit and debit rows in `transaction_entries`, ensuring mathematical balance reconciliation.

8.6 Threat Model and Educational Scope Limitations

IMPORTANT DISCLOSURE: WalletWise is engineered as an educational DBMS mini-project for college laboratory evaluations. It operates in a simulated demo environment and does not connect to real bank accounts, RBI/NPCI payment gateways, or real-money payment processors.

CHAPTER 9: CONCLUSION & REFERENCES

9.1 Project Contributions & Conclusion

The **"DIGITAL WALLET MANAGEMENT SYSTEM"** developed by **KISHOR M (Register Number: 711525BCS076)** at **Kalaignarkarunanidhi Institute of Technology, Coimbatore** proves that relational DBMS normalization (3NF), ACID transactions, double-entry ledger bookkeeping, and natural AI voice accessibility converge to safeguard electronic financial operations. Under the tagline **"Your Digital Wallet, Smarter & Safer"**, it proves that transactional integrity and technological elegance can be achieved simultaneously.

9.2 Future Enhancements

Future work will incorporate Zero-Knowledge Proofs for blind transaction receipts, multi-node Byzantine Fault Tolerant (PBFT) consensus clusters, and WebAuthn hardware biometric authentication.

REFERENCES (IEEE FORMAT)

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2020.
[2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2015.
[3] C. J. Date, *An Introduction to Database Systems*, 8th ed. Addison-Wesley, 2004.
[4] D. F. Ferraiolo, D. R. Kuhn, and R. Chandramouli, *Role-Based Access Control*, 2nd ed. Artech House, 2007.
[5] NIST, "Digital Identity Guidelines: Authentication and Lifecycle Management," *SP 800-63B*, 2020.
[6] OWASP Foundation, "OWASP Top Ten Web Application Security Risks," 2021. [Online]. Available: https://owasp.org
[7] React Documentation, "React: The Library for Web and Native User Interfaces," 2024. [Online]. Available: https://react.dev
[8] W3C Web Speech Working Group, "Web Speech API Specification," 2023. [Online]. Available: https://wicg.github.io/speech-api/

APPENDIX: DDL SCHEMA & API QUICK REFERENCE

```
-- Schema: users, wallets, transactions, transaction_entries, bill_payments, audit_logs
POST /api/auth/register | POST /api/auth/login | GET /api/wallet | POST /api/wallet/add-money
POST /api/transfers | POST /api/bills/pay | GET /api/transactions | GET /api/admin/dashboard
```